

GCP Data Engineering Interview & Practical Guide

Questions, Solutions, Code Snippets, BigLake, BigQuery Expectations & Cheat Sheet

Target Role:	GCP Data Engineer / Senior Cloud Data Engineer
Candidate Background:	5+ YOE Enterprise Data Pipelines, BigQuery, PySpark/Databricks, Airflow, SQL
Document Version:	v3 (Added BigLake DDL & BigQuery Data Quality Expectations)

SECTION 1: BIGQUERY ARCHITECTURE, LAKEHOUSE & DATA QUALITY

1. How do Partitioning and Clustering work together in BigQuery, and how would you optimize a multi-terabyte table for both cost and query performance?

Technical Deep-Dive

Candidate context: Leverages BigQuery and SQL extensively across financial and retail datasets.

Key Talking Points & Description:

- **Partitioning:** Divides the table into segments based on date/timestamp or integer range to eliminate scanning irrelevant partitions.
- **Clustering:** Sorts data based on up to 4 key columns within each partition.
- **Best Practice:** Partition by `transaction_date` and cluster by `customer_id` and `store_id` for granular query pruning.

Reference DDL / SQL Snippet:

```
-- Creating an optimized partitioned & clustered BigQuery table
CREATE OR REPLACE TABLE `my_project.finance.transactions`
(
  transaction_id STRING,
  customer_id STRING,
  transaction_date DATE,
  amount NUMERIC,
  store_id STRING
)
PARTITION BY transaction_date
CLUSTER BY customer_id, store_id
OPTIONS (
  partition_expiration_days = 1095,
  require_partition_filter = true
);
```

2. How do you set up a BigLake external table over GCS open data formats (Parquet/Iceberg) with row/column-level security?

Lakehouse Architecture

Candidate context: Expertise in combining data lake storage (GCS) with data warehouse compute (BigQuery).

Key Talking Points & Description:

- **BigLake Engine:** Connects BigQuery compute to external object storage (GCS/AWS S3/Azure Blob) via Cloud Resource Connections.
- **Governance:** Enforces fine-grained access policies (row-level filters and column masking) without granting users direct GCS storage permissions.

Reference BigLake DDL Snippet:

```
-- 1. Create External Lake Connection in BigQuery
-- (Executed via gcloud or terraform, resource name used below)

-- 2. Create BigLake Table over Parquet files in GCS
CREATE OR REPLACE EXTERNAL TABLE `my_project.lakehouse.sales_biglake`
WITH CONNECTION `us.my-gcs-connection`
OPTIONS (
  format = 'PARQUET',
  uris = ['gs://my-enterprise-datalake/sales_data/*.parquet'],
  hive_partition_uri_prefix = 'gs://my-enterprise-datalake/sales_data/'
);

-- 3. Apply Row-Level Security on BigLake Table
CREATE ROW ACCESS POLICY region_filter
ON `my_project.lakehouse.sales_biglake`
GRANT TO ("user:analyst@company.com")
FILTER USING (region = 'NORTH_AMERICA');
```

3. How do you implement automated Data Quality Assertions and Expectations directly inside BigQuery or SQL pipelines?

Data Quality & Governance

Candidate context: Background in operationalizing data quality checks in financial and retail production pipelines.

Key Talking Points & Description:

- **SQL Assertions:** Using BigQuery SQL procedural statements (`ERROR()`) or BigQuery DataFrames / Great Expectations integration.
- **Dataplex Data Quality:** Creating YAML rule definitions for automated column completeness, uniqueness, and custom SQL expressions.

Reference SQL Expectation / Quality Validation Snippet:

```
-- SQL Expectation Assertion Pattern for BigQuery / Airflow
DECLARE null_customer_count INT64;
DECLARE negative_amount_count INT64;

-- Check 1: Completeness Expectation
SET null_customer_count = (
  SELECT COUNT(1)
  FROM `my_project.finance.stg_transactions`
  WHERE customer_id IS NULL
);

-- Check 2: Validity Expectation
SET negative_amount_count = (
  SELECT COUNT(1)
  FROM `my_project.finance.stg_transactions`
  WHERE amount < 0
);

-- Raise Exception if Data Quality Expectations Fail
IF null_customer_count > 0 THEN
  ERROR(FORMAT("DQ Expectation Failed: Found %d null customer_ids", null_customer_count));
ELSEIF negative_amount_count > 0 THEN
  ERROR(FORMAT("DQ Expectation Failed: Found %d negative transaction amounts",
negative_amount_count));
END IF;
```

4. A query joining a 10TB transaction table with a 50MB dimension table in BigQuery is throwing memory limits / performance degradation. How do you resolve this?

Candidate context: Experience optimizing Spark/Databricks joins and applying similar principles in cloud warehouses.

Key Talking Points & Description:

- **Broadcast Join Mechanism:** BigQuery automatically broadcasts small tables, but skewed keys in the large table lead to worker memory imbalance.
- **Key Salting Technique:** Append a random integer (0–N) to skewed join keys to evenly distribute shuffle partitions across BigQuery slots.

Reference Query / Optimization Snippet:

```
-- Mitigating key skew using Salting Technique
WITH salted_transactions AS (
  SELECT *,
    CONCAT(customer_id, '_', CAST(FLOOR(RAND() * 5) AS STRING)) AS salted_key
  FROM `my_project.finance.large_transactions`
),
replicated_dimension AS (
  SELECT *,
    CONCAT(customer_id, '_', CAST(s.suffix AS STRING)) AS salted_key
  FROM `my_project.finance.customer_dim`,
  UNNEST([0, 1, 2, 3, 4]) AS suffix
)
SELECT t.transaction_id, t.amount, d.customer_name
FROM salted_transactions t
JOIN replicated_dimension d ON t.salted_key = d.salted_key;
```

SECTION 2: PIPELINE ORCHESTRATION & STREAMING ON GCP

5. How would you design an end-to-end real-time ingestion pipeline on GCP from Pub/Sub to BigQuery using Dataflow?

Architecture & Design

Candidate context: Strong background in Apache Spark/Databricks, Airflow, and Cloud ETL.

Key Talking Points & Description:

- **Architecture:** API/Event → Cloud Pub/Sub → Dataflow (Apache Beam) → BigQuery Storage Write API.
- **Beam Concepts:** Windowing (Tumbling/Fixed), Watermarking for late data, and PCollections.

Reference Python (Apache Beam) Snippet:

```
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions
from apache_beam.transforms.window import FixedWindows

def run_streaming_pipeline():
    options = PipelineOptions(streaming=True, save_main_session=True)
    with beam.Pipeline(options=options) as p:
        (
            p
            | "ReadFromPubSub" >> beam.io.ReadFromPubSub(subscription="projects/my-proj/
subscriptions/tx-sub")
            | "ParseJSON" >> beam.Map(lambda x: json.loads(x.decode("utf-8")))
            | "Window5Min" >> beam.WindowInto(FixedWindows(300))
            | "WriteToBigQuery" >> beam.io.WriteToBigQuery(
                table="my-proj:finance.live_transactions",
                write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND,
                create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED
            )
        )
```

6. How do you manage dependency orchestration and dynamic task generation in Cloud Composer (Airflow)? Orchestration

Candidate context: Hands-on Airflow experience across enterprise environments.

Key Talking Points & Description:

- **Cloud Composer:** Managed Apache Airflow running on GKE.
- **BigQueryInsertJobOperator:** Offloads heavy processing directly to BigQuery while keeping Airflow light.

Reference Airflow DAG Snippet:

```
from airflow import DAG
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator
from datetime import datetime

with DAG("gcp_bq_pipeline", start_date=datetime(2026, 1, 1), schedule_interval="@daily",
        catchup=False) as dag:

    run_transformation = BigQueryInsertJobOperator(
        task_id="bq_aggregate_daily_sales",
        configuration={
            "query": {
                "query": "CALL `my_project.finance.sp_daily_summary`()",
                "useLegacySql": False,
            }
        },
    )
```

SECTION 3: MIGRATION & GOVERNANCE

7. You need to migrate an existing PySpark batch data pipeline to GCP Dataproc / BigQuery. How do you write native PySpark code on GCS? Migration Scenario

Candidate context: Experience across multi-cloud (AWS, Azure, GCP) pipeline migration.

Key Talking Points & Description:

- **Cloud Dataproc:** Managed Spark/Hadoop clusters. Connects natively to GCS using `gs://` URLs.
- **BigQuery Connector:** Allows PySpark to write directly to BigQuery using indirect or direct methods.

Reference PySpark on GCP Snippet:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("GCP_Dataproc_Migration") \
    .getOrCreate()

df = spark.read.parquet("gs://my-data-lake-bucket/raw/transactions/year=2026/*")
clean_df = df.filter(df["status"] == "COMPLETED").groupBy("customer_id").sum("amount")

clean_df.write \
    .format("bigquery") \
    .option("writeMethod", "direct") \
    .option("table", "my_project.analytics.customer_spend_summary") \
    .mode("overwrite") \
    .save()
```

8. How do you implement data masking for PII columns using Policy Tags in BigQuery?

Governance & Security

Candidate context: Background in banking and financial services domain compliance.

Key Talking Points & Description:

- **Dataplex / Data Catalog Policy Tags:** Assign taxonomy tags to PII columns.
- **Dynamic Data Masking:** Automatically masks data based on IAM user privileges.

Reference SQL Policy Tag Definition:

```
CREATE TABLE `my_project.finance.customer_pii` (  
  customer_id STRING,  
  full_name STRING,  
  ssn STRING OPTIONS(policy_tags=['projects/my_project/locations/us/taxonomies/1234/  
categories/pii_high'])  
);
```

SECTION 4: GCP DATA ENGINEERING CHEAT SHEET

GCP Service	Primary Use Case	Key Technical Highlights & Snippets
BigQuery	Cloud Data Warehouse & Analytics	Serverless slot engine. Partitioning + Clustering. Enforce <code>require_partition_filter</code> . Supports SQL procedural assertions for Data Quality expectations.
BigLake	Unified Storage Engine (Data Lakehouse)	Query open formats (Parquet, Iceberg) in GCS via BigQuery WITH CONNECTION. Enforces column/row-level security without granting raw GCS permissions.
Cloud Dataflow	Batch & Real-time Stream Processing	Fully managed Apache Beam. Uses Watermarks, Windows (Tumbling/Session), and handles late data via Storage Write API.
Cloud Pub/Sub	Real-time Ingestion & Messaging	Global event ingestion buffer. Decouples producers/consumers with at-least-once delivery and dead-letter queues.
Cloud Composer	Workflow Orchestration	Managed Airflow on GKE. Offload heavy processing to BigQuery or Dataproc operators (<code>`BigQueryInsertJobOperator`</code>).
Cloud Dataproc	Managed Spark & Hadoop Clusters	Fast cluster startup (~90s). Ephemeral Clusters + Spot VMs reduce batch compute costs up to 80%. Connects to GCS (<code>`gs://`</code>).
Google Cloud Storage	Unstructured & Semi-structured Data Lake	Object storage. Tiering (Standard/Nearline/Coldline/Archive). Automated Lifecycle Management rules.
Dataplex / Data Catalog	Data Governance & Quality	Centralized metadata management, dynamic column PII data masking via Policy Tags, lineage tracking, and automated DQ rules.